**Trabalho Prático – Criptografia**

**Aluno:** Gabriel Costa da Silva

**Parte II – Projeto de Criptossistema Simétrico**

## 1 – Apresentação e Motivação

O presente documento visa apresentar toda a estrutura, análises e fundamentações de um criptossistema simétrico criado como exercício dos conceitos aprendidos na disciplina. Após uma análise dos paradigmas de criptografia sugeridos, mostraram-se mais interessantes as cifras de fluxo e ARX, pelos seguintes motivos:

- Keystream – permite a implementação de um método principal tanto para a cifragem quanto para a decifragem
- ARX – operações elementares da ULA que permitem tanto uma implementação sucinta quanto velocidade de execução
- Operação byte a byte – simplicidade do algoritmo, permitindo abstrair para somente a operação realizada num byte

Tendo esses pilares, e a forte inspiração no algoritmo já consolidado ChaCha20[1], seu principal diferencial vem de uma topologia tridimensional: a keystream é uma matriz tridimensional de ints (32 bits), simulando um cubo 3x3x3, e decorrências dessa diferença.

## 2 – Chave

A escolha de uma chave de 256 bits se deu a fim de gerar um nível adequado de segurança para o criptossistema e permitir comparações com o AES sem necessidade de especulações por diferenças no tamanho das chaves.

## 3 – Nonce e Contador

O nonce tem a função de impedir que, apesar de usarem a mesma chave, duas mensagens diferentes repitam uma keystream durante sua cifragem. Dessa forma, é adicionado um número aleatório (Number used once – nonce) como entrada para gerar cada keystream. Tratando-se de números aleatórios, existe a chance (Paradoxo do Aniversário) de que se repita o nonce, para contornar isso, foi escolhido o tamanho de 192 bits para o nonce, a fim de que a chance estatística de uma repetição ocorrer seja desprezível.

Outro número inserido na geração do keystream é o contador. Caso todos os blocos fossem cifrados utilizando a mesma keystream, um XOR entre dois blocos anularia a keystream, gerando um XOR entre o texto claro desses dois blocos, o que quebraria a segurança do sistema. Dessa forma, insere-se um contador ao gerar a keystream, a fim de que ela seja diferente para cada bloco. O uso de 1 int para o contador garantiria a inexistência de colisões em $2^{32} \cdot 108$ bytes $\approx 463$ Gigabytes. Assim, 2 ints (1 long), que garantem $2^{64} \cdot 108$ bytes $\approx 2$ Zettabytes de diferenciabilidade, são mais recomendados para sistemas robustos e foi o tamanho implementado.

## 4 – Keystream

- **Inicialização**

A chave, o nonce e o contador são divididos, respectivamente, em blocos de 8, 6 e 2 inteiros e então são distribuídos com outros 11 inteiros constantes no Cubo.

Na etapa de ARX os elementos centrais são aqueles que mais espalham entropia para os demais elementos do sistema. Dessa forma, estes devem ser ocupados pela chave a fim de reduzir a previsibilidade do sistema. Assim, o cubo é iniciado nas seguintes camadas:

Topo (Y=0):

| | X = 0 (Esquerda) | X = 1 (Centro) | X = 2 (Direita) |
|---|---|---|---|
| Z = 0 (Frente) | Contador(High) | Constante 0 | Nonce 0 |
| Z = 1 (Meio) | Constante 1 | Chave 1 | Constante 2 |
| Z = 2 (Trás) | Nonce 2 | Constante 3 | Nonce 4 |

Equador (Y=1):

| | X = 0 (Esquerda) | X = 1 (Centro) | X = 2 (Direita) |
|---|---|---|---|
| Z = 0 (Frente) | Chave 7 | Chave 5 | Constante 4 |
| Z = 1 (Meio) | Chave 3 | Chave 0 | Chave 4 |
| Z = 2 (Trás) | Constante 5 | Chave 6 | Constante 6 |

Base (Y=2):

| | X = 0 (Esquerda) | X = 1 (Centro) | X = 2 (Direita) |
|---|---|---|---|
| Z = 0 (Frente) | Nonce 1 | Constante 7 | Nonce 3 |
| Z = 1 (Meio) | Constante 8 | Chave 2 | Constante 9 |
| Z = 2 (Trás) | Nonce 5 | Constante 10 | Contador(Low) |

- **Rota e Rodadas**

Para aplicar o paradigma ARX, deve-se escolher em quais elementos ele será aplicado. Seguindo a topologia da keystream, os elementos serão embaralhados em trios, sendo estes trios divididos em 4 conjuntos disjuntos de 9 trios:

- ► Conjunto 1 – Variação Ortogonal em X
- ► Conjunto 2 – Variação Ortogonal em Y
- ► Conjunto 3 – Variação Ortogonal em Z
- ► Conjunto 4 – Diagonais Toroidais – além das diagonais internas, usam-se também diagonais tomando que a matriz é um toro, ao utilizar aritmética modular.

Essa divisão tem dois objetivos:

1. Eliminar dependência de dados (Cada elemento é acessado uma única vez em cada conjunto)
2. Uniformizar a distribuição da entropia

Quanto ao número de rodadas, deve-se analisar a velocidade de difusão da entropia. A maior distância entre dois elementos do cubo é 2 (distância topológica no grafo de roteamento), e em uma rodada um elemento afeta todos a uma distância de 1 dele e parcialmente aqueles de distância 2. Ou seja, ao final de 2 rodadas, um elemento já espalhou entropia para todos os outros elementos do cubo. Porém, a única operação não-linear do núcleo ARX é a soma, por conta do carry (1 bit), por isso, são necessárias de 12 a 16 rodadas para garantir o grau de confusão necessário para a robustez do sistema. Dessa forma, a implementação do projeto foi realizada com 16 rodadas.

- **ARX**

Dado um trio (a, b, c) da rota, são realizadas as seguintes operações:

| | | | |
|---|---|---|---|
| 1º | $a = a + b$ | $c = c \oplus a$ | $c = c \lll 16$ |
| 2º | $a = a + c$ | $b = b \oplus a$ | $b = b \lll 12$ |
| 3º | $a = a + b$ | $c = c \oplus a$ | $c = c \lll 8$ |
| 4º | $a = a + c$ | $b = b \oplus a$ | $b = b \lll 7$ |

Cada elemento tem seu valor alterado 4 vezes, para não gerar assimetria na difusão.

- **Davies-Meyer**

Como todos os processos da etapa ARX são perfeitamente invertíveis, utiliza-se a Construção Davies-Meyer para tornar a criação da keystream uma Função Pseudoaleatória e quebrar a reversibilidade. Por isso, a última etapa da criação é uma soma do estado inicial após as 16 rodadas com o próprio estado original.

**5 – Processamento de dados**

O processamento central dos dados (válido para cifrar e decifrar), recebe a mensagem, a chave, e o nonce como, respectivamente, um array de bytes, e dois arrays de ints.
Então, cada byte da mensagem é tratado separadamente da seguinte maneira: descobre-se e coleta-se o byte dos $27 \cdot 4 = 108$ da keystream referente a ele, e , então, aplica-se um XOR no byte da mensagem com o byte referente da keystream e guarda-o no array de Saída.
A cada 108 Bytes da mensagem, uma nova keystream é gerada com o contador incrementado.

**6 – Funcionamento do Sistema**

- **Cifragem**

Gera-se um nonce aleatório com um gerador seguro, processa-se a mensagem, chave e nonce usando o processamento de dados, concatena-se o nonce utilizado no início da mensagem e a envia-se. Como a cifragem de um byte ocorre em $O(1)$ em relação ao tamanho da entrada, a complexidade do processo de cifragem é de $O(n)$, sendo $n$ o tamanho da mensagem em bytes. Além disso, como o custo espacial da keystream também é $O(1)$, a cifragem tem custo espacial de $O(n)$ também, por conta da alocação do vetor de saída.

- **Decifragem**

Recebe-se o texto cifrado, retira-se o nonce concatenado do início, passam-se o resto da mensagem junto com a chave e o nonce recuperado pelo processamento de dados, e envia-se o resultado. A decifragem possui as mesmas complexidades assintóticas e complexidade espacial da cifragem.

**7 – Análise de Desempenho**

- **Operações por byte**

► Cada trio opera 4 adições, 4 XOR e 4 rotações, totalizando 12 operações.
► Cada rodada opera em 36 trios, o que leva a $36 \cdot 12 = 432$ operações.
► 16 rodadas totalizam $16 \cdot 432 = 6912$ operações.

► Davies-Meyer realiza 27 operações.

Dessa forma, gerar uma keystream exige $6912 + 27 = 6939$ operações, para cifrar 108 bytes, levando a uma relação de $\frac{6939}{108} = 64,25$ operações por byte gerado, o que é um custo de ciclos de instrução muito mais pesado que as 15,25 operações do ChaCha20[2] ou as 22 do AES em software[3], apresentando um custo computacional elevado.

- **Paralelismo não implementado**

O código pode ser paralelizado, uma vez que, dadas a chave, o nonce e a mensagem, cada keystream depende somente do seu contador, que é previsível. Porém, o código não foi implementado dessa forma, mas sim de forma estritamente sequencial.

**8 – Análise de Segurança**

- **Pontos Fortes:**

► Imunidade a Canais Laterais

Como todas as operações do núcleo ARX são realizadas em tempo constante, não é possível usar o tempo de execução do sistema como vetor de ataque.

► Imunidade a Two-Time Pad

A implementação do nonce seguro de 192 bits torna estatisticamente impossível a ocorrência de Two-Time Pad.

► Confusão e difusão

Como apresentado na seção sobre a quantidade de rodadas, a difusão completa é atingida em 2 rodadas.

Sobre a confusão, a adição (carry) é a única operação que acrescenta não-linearidade ao sistema, e ela o faz multiplicando a chance de sucesso de previsão de seu resultado por $2^{-2}$, que é a probabilidade máxima de transição diferencial na soma de inteiros, tomando a linearização do carry. Considerando o melhor cenário para um atacante, que seria reter as mudanças a somente uma Trilha Esparsa, o menor objeto desse tipo na topologia do cubo seria um quadrado 3x3. Como cada elemento desse quadrado realizaria pelo menos uma soma, a dificuldade de previsão após uma rodada cai para $(2^{-2})^9 = 2^{-18}$, dessa forma, o número x de rodadas é tal que $2^{-18x} < 2^{-256}$ para garantir 256 bits de segurança. Assim, $x \approx 14,22$.

Portanto, o uso de 16 rodadas na geração da keystream se justifica a fim de garantir altíssima difusão e o grau necessário de confusão para resistir a Criptoanálise Diferencial e Linear, com uma sobra estratégica.

- **Ponto Fraco:**

► Maleabilidade

O Criptossistema não cobre a autenticação necessária para oferecer integridade, sendo possível manipulá-lo caso o atacante saiba a estrutura do texto claro.

## 9 – Comparação com o AES

- **Tamanho de chave**

Ambos os criptossistemas usam chaves de 256 bits.

- **Tamanho de bloco**

O AES usa um de 128 bits, enquanto o implementado, possui um bloco de $108 \cdot 8 = 864$ bits, referente ao seu estado interno (keystream).

- **Velocidade**

Para comparar a velocidade do AES (utilizando AES-NI e não utilizando) com a da cifra projetada,foi utilizado um Benchmark com as seguintes características:

► Payload de 64KB – permite que os dados fiquem dentro da cache
► Aquecimento de 50.000 iterações – força o uso do compilador C2 (uso de AES-NI)
► Medição com 100.000 iterações – Aproximadamente 6.2GB

Chegando aos seguintes resultados:

► AES (AES-NI): 2003.95 MB/s
► AES (Software): 64.16 MB/s
► Cubo (Software): 10.78 MB/s

O AES (Software) ser 6 vezes mais rápido é justificado pela diferença de operações por byte cifrado e pelas operações não elementares utilizadas no mapeamento do byte dentro da keystream.

- **Consumo de memória**

Enquanto o AES pode consumir até 4KB de memória para armazenar suas T-Tables, a cifra implementada usa apenas os 108 Bytes da keystream durante sua execução.

- **Paralelização**

► Blocos

Na cifragem de uma mensagem através da cifra do Cubo, a diferença entre a keystream de dois blocos se dá pelo contador, que é previsível e perfeitamente paralelizável, por mais que a implementação tenha sido feita sequencialmente.
Portanto, tanto a cifra projetada quanto o AES-CTR empatam no quesito de paralelização de blocos.

► Dentro do bloco

Como em cada conjunto de trios da rota, cada elemento é referenciado apenas uma vez, não há conflito de dados, permitindo a paralelização da execução de cada conjunto.
Nesse quesito, a cifra ganha teoricamente do AES-CTR.

- **Difusão e confusão**

  ► Difusão

O AES atinge difusão total em apenas 2 rodadas, enquanto a cifra projetada também o faz, como argumentado na escolha do número de rodadas. Nesse aspecto, é um empate teórico.

  ► Confusão

O AES atinge altíssima não linearidade pelo uso das S-Boxes, chegando à confusão total muito rapidamente, enquanto a cifra projetada tem somente a adição como operação que gera não linearidade, precisando de muitas mais rodadas para atingir um nível adequado de segurança.

- **Resistência teórica a ataques**

  ► Criptoanálise Diferencial e Linear

Enquanto o AES atinge confusão e difusão magistralmente com suas S-Boxes e operações lineares, a cifra projetada faz uso extensivo de rodadas para garantir a segurança. Dessa forma, a cifra foi projetada para ter resistência suficiente à Criptoanálise Diferencial e Linear.

  ► Ataque de Chaves Relacionadas

Enquanto o AES resolve esse problema utilizando Key Schedule, a cifra projetada lida com esse tipo de ataque através do tamanho massivo de seu estado interno e na quantidade de rodadas.

  ► Colisões de Estado interno

O estado interno do AES possui 128 bits, gerando riscos a partir de $2^{64}$ blocos cifrados, enquanto o estado interno da cifra projetada tem 864 bits, gerando riscos a partir de $2^{432}$ blocos cifrados, o que é intratável.

  ► Canais Laterais

O AES em software pode ter o tempo de acesso às T-Tables usado como vetor de ataque para descobrir a senha, enquanto a cifra projetada tem em sua concepção a imunidade a esse tipo de ataque, tendo em vista que as operações do núcleo ARX têm tempo constante.

- **Simplicidade de implementação**

O AES tem um código relativamente extenso, utilizando de conceitos abstratos como corpos estendidos, é uma implementação complexa, ainda mais considerando medidas de proteção aos ataques temporais mencionados na comparação de ataques por Canais Laterais.

Enquanto isso, a cifra projetada tem em sua própria concepção o intuito de ter uma fácil implementação. Ela usa um algoritmo central no qual se baseiam as operações de cifragem e decifragem, além do uso exclusivo de operações elementares.

- **Escalabilidade**

► Ambiental

Escalando para cima, o AES possui desempenho amplamente superior, tendo em vista o esforço de produtoras de processadores em implementar o AES-NI, enquanto a cifra projetada teria que ser implementada usando o máximo de paralelização possível, comentada em outro tópico.

Analisando o caso de aparelhos mais restritos, a falta de AES-NI leva a implementação a ser software, requisitando os 4KB de memória e utilizando-a extensivamente, que podem ser um alto preço a se pagar. Enquanto isso, a cifra projetada aloca menos memória do sistema e usa somente operações já previstas na microarquitetura.

► Paramétrica

O AES, por ser o algoritmo consagrado que é, não aceita mudanças em sua estrutura, por isso, a escalabilidade paramétrica dele é nula (Salientando que isso não é um problema do AES). Enquanto a cifra proposta, como projeto de software, aceita modificações e está pronta para tanto, tendo em vista que a keystream é inicializada com 13 elementos constantes, que poderiam ser reduzidos a 7, caso necessário.

► Carga

O AES, por ter blocos menores que a cifra proposta, evita desperdícios de processamento ao lidar com micro-mensagens, levando vantagem na escalabilidade de carga.

**10 – Testes Funcionais**

Teste 1

**Texto claro:** Mensagem de Teste pequena

**Chave:** [ 0xAD15C507, 0xCE38EFE2, 0xC3A7109C, 0xB068C3EA, 0xEB6D621F, 0xCD9E4CA6, 0x067525D8, 0xAAAFAE86 ]

**Texto cifrado:**
D886BB752A9F05E747CB2AA092A1E45F382
376E2242757148191D2303FBEA0A9D8536FC5
D510E4CBBA0FA489D0A89C58D4

**Texto decifrado:** Mensagem de Teste pequena

Teste 2

**Texto Claro (Fragmento):**
"INICIO_DO_STRESS_TEST: Este é um texto projetado [...] antes de atingir o FIM_DO_TESTE." (Tamanho total: 868 bytes)

**Chave:** [ 0xB40C22FB, 0xD09F4CFE, 0x484D9146, 0x46665B41, 0x7B081C23, 0xD8283DC6, 0x8D5B4E91, 0x2470D1F9 ]

**Texto Cifrado (Fragmento):** [Nonce: EC FE 39 75 49 ...] 41 35 8F B8 71 88 ... [Aprox. 800 bytes suprimidos] ... 7C 05 40 4A 54

**Mensagem Decifrada (Fragmento):**
"INICIO_DO_STRESS_TEST: Este é um texto projetado [...] antes de atingir o FIM_DO_TESTE."

Os testes completos encontram-se no arquivo testes.txt

## 11 – Uso de Inteligência Artificial

- **Implementação**

O Gemini 3.1 Pro foi utilizado como auxiliar no desenvolvimento da implementação, gerando funções auxiliares para o funcionamento do projeto.

- **Teoria**

O mesmo modelo de IA também foi utilizado para o aprofundamento nas temáticas das cifras de fluxo, do paradigma ARX, da técnica Davies-Meyer e de conceitos criptográficos importantes para uma boa implementação.

## 12 - Considerações finais

- **Limitações da Linguagem**

O desenvolvimento do projeto na linguagem de programação Java foi positivo pelas qualidades intrínsecas à orientação a objetos. Porém, também impactou significantemente o desempenho da aplicação:

- ► Matrizes de 3 dimensões não são dados adjacentes na memória, mas sim ponteiros para ponteiros para dados, acarretando um overhead de processamento e de memória, totalizando 336 bytes alocados na memória para gerar uma keystream, em vez de 108, além de quebrar a localidade espacial da qual a cache se beneficia, elevando os cache misses.
- ► A fim de evitar Segmentation Fault, a linguagem verifica os bounds de cada dimensão da keystream, trazendo mais um overhead de processamento.
- ► Além disso, ao não permitir o gerenciamento manual de memória, fica a cargo do garbage collector remover da memória principal keystreams já usadas, o que pode levar a um desperdício de memória quando duas ou mais existirem ao mesmo tempo.

- **Topologia não ideal**

O uso topológico de um cubo 3x3x3 numa cifra foi um exercício geométrico muito educativo, porém propriedades ineficientes desse modelo se mostraram claramente nos seguintes aspectos:

- ► preenchimento de 40,7% da keystream com constantes, quando poderia ser de 25%.
- ► operações como /9 e %3 presentes ao processar cada byte da mensagem, afetando o desempenho da cifra.

**13 – Bibliografia**

[1] BERNSTEIN, Daniel J. ChaCha, a variant of Salsa20. Em: *Workshop on Record of the SASC*, Lausanne, Suíça, v. 8, n. 1, p. 3-5, 2008. Disponível em: https://cr.yp.to/chacha/chacha-20080128.pdf. Acesso em: 29 jun. 2026.

[2] NIR, Y.; LANGLEY, A. ChaCha20 and Poly1305 for IETF Protocols. *Request for Comments: 8439*, Internet Engineering Task Force (IETF), maio 2018. Disponível em: https://datatracker.ietf.org/doc/html/rfc8439. Acesso em: 29 jun. 2026.

[3] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (US). *Advanced Encryption Standard (AES)*. Federal Information Processing Standards Publication (FIPS PUB) 197. Gaithersburg, MD: NIST, nov. 2001. Disponível em: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf. Acesso em: 2 jul. 2026.

[4] DAEMEN, Joan; RIJMEN, Vincent. *The Design of Rijndael: AES - The Advanced Encryption Standard*. 1. ed. Berlim: Springer-Verlag, 2002.